

Gebze Technical University

CSE 341 — Concepts of Programming Languages · Spring 2026

Project Part 1 — AI Usage Journal (D4)

ScoreScript

Student Number: 230104004090

Name: FATİH EMRE OĞAN

Submission Date: 8 May 2026

This journal documents my substantive interactions with AI tools during the development of ScoreScript for CSE 341 Part 1. Each entry records the prompt, the response, what I accepted or rejected, and my critical evaluation. Entries are dated across the working period of Part 1 (2–8 May 2026).

Entry #1 — Domain Selection

Entry #1	Date: 2026-05-03 Phase: Design AI Tool: Claude Sonnet 4.6
Goal	Choose a suitable domain for my DSL project.
Prompt	I have a CSE 341 Programming Languages course project. I need to design a small DSL. I'm thinking about something related to student grading. What domain would you suggest and why?
Response	AI suggested a DSL called GradeLang where you define courses, grading components like midterm and final, and rules for passing or failing. It used course and rule blocks as the main domain-specific constructs.
Accepted	The general direction of a grade management DSL was good domain is appropriate for the project and different from handout examples.
Rejected / Modified	The course component, rule, flag structure was too complex. Those nested blocks would make the parser really hard to write. I wanted something simpler where the student is the main thing, not the course.
Errors Caught	AI gave a design that looks more like a config file than a real programming language. The handout shows a different style and AI missed that. It also ignored how hard it would be to implement.
Reflection	<i>AI's first suggestions are usually too complex. Simpler designs are easier to implement and test.</i>

Entry #2 — Adding Functions and Variables to the Grammar

Entry #2	Date: 2026-05-03 Phase: Design AI Tool: Claude Sonnet 4.6
Goal	Add user-defined functions and variable declarations to the grammar.
Prompt	The declaration section is missing two things. I also want user defined functions like: <pre>func letterGrade(avg: float) -> string { if avg >= 90 { return "AA" } else if avg >= 85 { return "BA" } else { return "FF" } }</pre> And variable declarations like <code>int x = 5</code> or <code>float threshold = 60.0</code>. Both should be in the declaration section before statements. Update the grammar according to what I said.
Response	AI updated the grammar with <code>func_decl</code> and <code>var_decl</code> in the declaration section. It also added a proper expression grammar with arithmetic operators and function call expressions.
Accepted	The general <code>func_decl</code> and <code>var_decl</code> structure. The expression grammar with proper layers for arithmetic was also a good addition.
Rejected / Modified	AI used a separate <code>func_body</code> with <code>func_stmt</code> rule. I later merged this with the general <code>stmt</code> rule to avoid duplication. Also operation syntax was still wrong: "average IDENT IDENT" and "rank IDENT by average" . I simplified both later.
Errors Caught	AI forgot to add <code>void</code> as a return type even though functions that only print and don't return a value are needed. Also the expression grammar was missing boolean operators : no <code>&&</code> , <code> </code> , or <code>!</code> anywhere.
Reflection	AI remembered our previous conversation but missed some things I did not explicitly ask for. I had to ask for <code>void</code> and boolean operators separately in later prompts.

Entry #3 — E1: EBNF Grammar Generation from Description

Entry #3	Date: 2026-05-03 Phase: Design AI Tool: Claude Sonnet 4.6
Goal	Ask AI to generate an EBNF grammar from a short description of ScoreScript and compare it with my own design.
Prompt	I want to keep it simpler. The course structure feels too complex for a first DSL. I want something where the student is the main entity. So I define students with their grades, group them and then do things like compute averages, apply a curve, show a transcript, rank them. Something like: student Fatih { math: 85, science: 90 } group ClassZ23 = [Fatih] curve ClassZ23 by 5 transcript Fatih. Can you suggest a design like this with a short example?
Response	AI produced a rough EBNF grammar for a language it called RosterLang. The grammar had student declarations, group declarations, operations and if statements. It also suggested three primitive types: int, bool, string.
Accepted	The general structure of declarations and operations was a good starting point. The observation was correct , every statement starts with a unique keyword.
Rejected / Modified	Several things were wrong compared to my final design. AI wrote average IDENT IDENT and rank IDENT by average . I simplified both to just average IDENT and rank IDENT. Student had no id, firstName, lastName fields. No float type was included. Comment syntax used # instead of //. There was no expression grammar ,only arithmetic was mentioned, no boolean operators or precedence layers. The <program> rule mixed declarations and statements instead of separating them.
Errors Caught	Operator precedence was completely missing . AI gave no precedence layers at all. My final grammar has 7 layers from unary to . AI also did not include float type even though averages naturally return decimal values. The # comment syntax conflicted with our Java implementation convention.
Reflection	AI's first EBNF is a useful starting point but misses many details. The biggest gap was operator precedence . AI produced a flat expression grammar with no layering. I had to ask for boolean operators and proper precedence separately in later prompts.

Entry #4 — Final Grammar Verification

Entry #4	Date: 2026-05-03 Phase: Design AI Tool: Claude Sonnet 4.6
Goal	Ask AI to write the complete final grammar and verify it matches my design decisions.
Prompt	Write the complete final grammar one more time with all the changes we made.
Response	AI produced the complete final EBNF grammar including all design decisions: declarations before statements, fixed student field order, void separated from value types, seven expression precedence layers, for loop with mandatory student keyword and a summary table of every decision with Sebesta references.
Accepted	The overall grammar structure was correct and matched my design. The summary table with Sebesta references was useful for D1.
Rejected / Modified	The <code><expr> ::= <or_expr></code> wrapper was unnecessary . I start directly from <code><or_expr></code> in my implementation. Also <code>func_call_expr</code> only used IDENT but in my design domain keywords like <code>average()</code> can also be called as expressions.
Errors Caught	AI wrote <code><expr> ::= <or_expr></code> as an extra wrapper rule with no purpose. This adds an unnecessary layer to the grammar. I removed it in my final version. AI also missed that domain keywords like <code>average</code> and <code>transcript</code> should be callable inside expressions.
Reflection	Asking AI to produce the final version of something is a good way to catch wrongs. Comparing the output with my own notes revealed two small gaps I had not noticed before.

Entry #5 — Implementation: Lexer, Parser and AST

Entry #5	Date: 2026-05-03 Phase: Lexer / Parser AI Tool: Claude Sonnet 4.6
Goal	Ask AI to implement the complete ScoreScript lexer, parser and AST in Java based on our final EBNF grammar.
Prompt	Now I want to implement ScoreScript in Java. Based on our final EBNF grammar give me all the files: - TokenType.java - Token.java Lexer.java // comments no package - AST node classes - Parser.java - Main.java with --dump-ast support No package declarations. Parser should reject invalid programs with line numbers in error messages.
Response	AI generated all six Java files. It implemented a recursive descent parser, dedicated token types for student field keywords, FLOAT before INT ordering in the lexer, // comment handling, and --dump-ast flag.
Accepted	The overall architecture ,separate files for TokenType, Token, Lexer, AST, Parser, Main. The recursive descent approach matches our grammar structure. FLOAT before INT ordering was correctly implemented. Student field keywords as dedicated token types was a good decision.
Rejected / Modified	The comment syntax used # instead of // . I corrected this. AI also had a package declaration which I removed. The average and rank operations still had wrong syntax in the sample program .AI used average(ClassZ23, math) which is not valid in our grammar.
Errors Caught	AI used # for comments instead of // which we had already decided. It also generated a sample program with average(ClassZ23, math)(We dont want). AI caught and fixed this itself during testing, which shows it does self-correct but only when it actually runs the code.
Reflection	AI is good at generating repetitive code quickly but does not always remember every design decision from the conversation. Testing the output immediately is important. Bugs only appear when you actually run the code.

Entry #6 —Writing Example Programs and Finding a Parser Bug

Entry #6	Date: 2026-05-04 Phase: Testing AI Tool: Claude Sonnet 4.6
Goal	Ask AI to write 3 example ScoreScript programs that together cover all language constructs.
Prompt	I need 3 example ScoreScript programs to test my parser. They should be different from each other and together cover: student declarations, groups, functions with if/else, for loops, curve/average/rank/transcript and variable declarations.
Response	AI generated three programs covering all constructs. During testing a bug appeared . Writing a function call like printLabel(92.0) as a statement caused a parser error. AI fixed it by adding a CallStmt node and checking whether the next token after IDENT is (or =.
Accepted	The three programs and the CallStmt fix. The fix was correct . When the parser sees IDENT followed by (, it should treat it as a function call statement, not an assignment. This is a real design gap I had not noticed.
Rejected / Modified	Nothing major rejected. I removed comments from the source code before putting it in D3 to keep it cleaner.
Errors Caught	The parser could not handle function calls used as statements. AI only found this bug by actually running the code, not by reading the grammar. This shows that testing is always necessary even if everything looks correct on paper.
Reflection	Writing test programs is a good way to find grammar gaps. A bug I did not notice during design showed up immediately when I ran a real program. AI is useful for generating test cases but you still need to verify the output yourself.

Entry #7 — Testing Error Programs and Verifying Parser Error Messages

Entry #7	Date: 2026-05-04 Phase: Testing AI Tool: Claude Sonnet 4.6
Goal	Test all 5 malformed programs and verify that error messages and line numbers are correct.
Prompt	I tested all 5 error programs and all work correctly. All error messages show the right line numbers
Response	AI confirmed the milestone and summarized what is completed: EBNF grammar, lexer, parser, AST, --dump-ast flag, 3 valid programs and 5 malformed programs. It also listed what remains for Part 1 deadline.
Accepted	The summary was accurate. All error messages and line numbers matched what I expected
Rejected / Modified	Nothing rejected. I ran each error program myself in the terminal and verified the output before accepting.
Errors Caught	No errors in this session. However I noticed that error5 initially showed no output .I had not pressed Enter in the terminal. This was my own mistake, not an AI error.
Reflection	Running the programs myself was important. Even though AI said everything worked, I verified each error message and line number personally. Trusting AI output without testing is not enough.

Entry #8 — Generating D1 Design Specification Document

Entry #8	Date: 2026-05-05 Phase: Writing AI Tool: Claude Sonnet 4.6
Goal	Ask AI to generate a D1 document template with §4.2 and §4.3 fully written and placeholders for §4.1 and §4.6.
Prompt	Write D1 Design Specification document for ScoreScript as a Word docx file. Include these sections: 4.1 Language Overview half page, 4.2 Lexical Structure half page, 4.3 Syntax in EBNF 2 pages, 4.6 Names Binding Scope Lifetime 1 page. For 4.1 and 4.6 write placeholder text like [STUDENT WILL WRITE THIS PART] because I should write those parts myself in my own words. Only generate 4.2 and 4.3 fully
Response	AI generated a complete Word document with cover page, fully written §4.2 and §4.3, and placeholder sections for §4.1 and §4.6 with bullet-point prompts explaining what to write.
Accepted	The §4.2 token tables and §4.3 EBNF were correct and matched our final grammar. The document structure and formatting were professional.
Rejected / Modified	The IDENT pattern was wrong AI wrote <code>[a-zA-Z][a-zA-Z0-9_]*</code> with leading underscore allowed. I corrected it to <code>[a-zA-Z][a-zA-Z0-9]*</code> because we had specifically decided leading underscores are not allowed for readability reasons.
Errors Caught	AI forgot that leading underscores are not allowed in identifiers. This was a deliberate design decision we made earlier in the conversation but AI did not carry it over to the document. I caught it by comparing the document with my own design notes.
Reflection	I wrote §4.1 and §4.6 myself in my own words. AI-generated technical sections like EBNF and token tables saved time but the design rationale sections needed to be written by me. Always check AI output against your own design notes.

Entry #9 — Final Submission Check

Entry #9	Date: 2026-05-07 Phase:Submission AI Tool: Claude Sonnet 4.6
Goal	Verify all Part 1 deliverables are complete before converting to PDF.
Prompt	I finished all the files for Part 1. Before I convert everything to PDF do a final check. I have D1 D3 D4 D5 as docx files. D2 is a zip with src folder examples folder Makefile and README. List what each file should contain for Part 1 and I will confirm everything is in order.
Response	AI read all files and confirmed everything is correct. It listed all deliverables with checkmarks and reminded me about the filename convention with student ID prefix.
Accepted	The checklist was accurate. All deliverables confirmed complete.
Rejected / Modified	Nothing rejected.
Errors Caught	AI reminded me about the filename convention . Files must start with student ID (230104004090_D1_P1.pdf etc.). I might have missed this without the check.
Reflection	Doing a final check before submission is worth the time. AI caught the naming convention detail and confirmed everything was in order.

AI Conversation Reference

The AI interactions documented in this journal were conducted in the following Claude conversation:

Chat Link: <https://claude.ai/share/7ab7b34e-a816-4d7c-9a4a-4549d57a47a6>

AI Tool: Claude Sonnet 4.6

Platform: Claude.ai